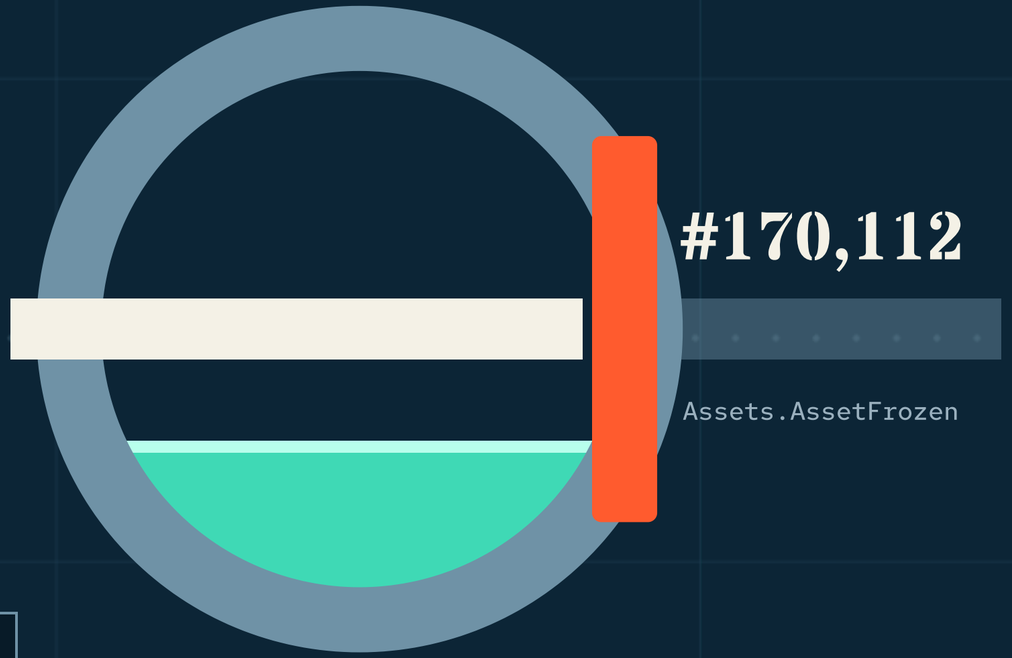
10 09 08 07 06 05 04 03 02 01

WHEN THE VAULT RUNS SHORT, THE TEST DOLLAR STOPS MOVING.

BALLAST a reserve-gated circuit breaker for a custodial test dollar on Portaldot.



re-run read-only 2026-10-31 · evidence/check-receipts-v3-20261031T034000Z.txt

```
$ python scripts/check_receipts.py --profile v3 | grep '^\[v3\]'
[v3] 261/261 receipts confirmed from 23 run(s)
[v3] all confirmed rows at or below the finalized head #261558
```

On the Portaldot V3 testnet, 2026-10-24: watcher W2 froze asset 4401 in extrinsic `170112-1`.
Reserve: local anvil chain, NOT Sepolia. Deck 2026-10-25.

THE PROBLEM

RESERVES DRAIN. NOBODY SEES.

As far as we found, Portaldot has no stable unit of account. The realistic first step is a custodial wrapped dollar, and when its reserves drain, every holder keeps trading an IOU that is already worth less than its face value.

01

104.00

in supply: 54.00 pUSDC-test (asset 4401) + 50.00 control asset (4402), matched 1:1 by test tokens in the vault.

02

−31.20

The custodian calls `release()`: 30% of the reserve leaves the vault.

03

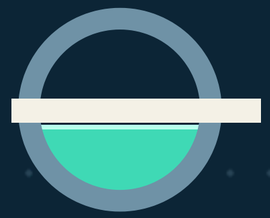
72.80

Reserves left. Supply still says 104.00 unless something on Portaldot acts.

Figures from Ballast's own V3 drill (`evidence/roles-drill-v3-20261024T045513Z.json`; reserve on a local anvil chain, NOT Sepolia). The person it protects: a dApp builder deciding whether checkout may accept pUSDC-test.

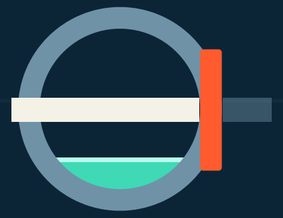
10 09 08 07 06 05 04 03 02 01

FOUR ROLES, THREE MULTISIGS



LEVEL

reserves ≥ supply · asset Live · transfers work



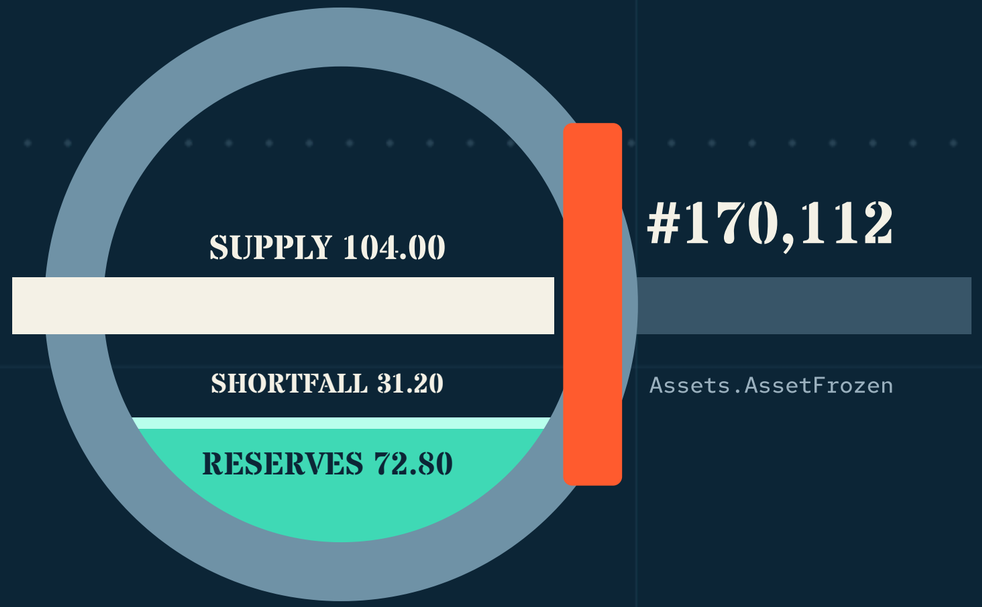
TRIPPED

reserves < supply · asset Frozen · `Assets.transfer` refused

ROLE	HELD BY	WHAT IT MAY DO
FREEZER	`M_watch` 1-of-2 watchers W1, W2	Any one watcher freezes the asset when reserves < supply. Its thaw attempts fail with `Assets.NoPermission`.
ISSUER	`M_mint` 2-of-2 operator A, reserve bot B	The bot co-signs a mint only if *finalized* reserves cover supply plus the amount, and never while frozen.
ADMIN	`M_admin` 2-of-3 A, A2, B	The bot co-signs a thaw only after finality passes the drain block and reserves cover supply again.
RECORD	bot B remarks + EVM mirror	`BALLAST/1` attestations on chain, mirrored into `ReserveOracle`; `ballast verify` re-reads both chains.

Roles are set with `Assets.set_team`; the reserve check itself runs off chain, in the bot and the watchers.

RESERVES SANK. ASSET FROZE.



anvil #1622
RESERVE CHAIN

Custodian drains 30% (31.20) with `release()`.

170112-1
V3 · WATCHER W2

`M_watch` freezes asset 4401 (`Assets.freeze_asset`, Ok). The reason rides in the same extrinsic:

```
BALLAST/1|freeze|a=4401|reason=deficit|s=104000000|
r=72800000|p=anvil|eb=1622|eh=0x3987686a…
```

170114-1
V3 · A HOLDER

`Assets.transfer` 1.00 → `Err(Assets.AssetNotLive)`

170115-1
V3 · WATCHER W1

A watcher tries `thaw_asset` → `Err(Assets.NoPermission)`: the freezer role cannot thaw what it froze.

Drain to freeze included: 12.22 s (wall clock, one machine). Gauge = the freeze remark: s=104000000 (supply of asset 4401 + control asset 4402), r=72800000, 6 decimals. Reserve: local anvil chain, NOT Sepolia.

RECORDED PROOF · THE ROLE DRILL ON V3

GATES SIGNED, GATES REFUSED

10 09 08 07 06 05 04 03 02 01

roles-drill-v3-20261024T045513Z.json · bot B decisions (… = cut) · reserve: local anvil, NOT Sepolia

```
$ ballast bot cosign-mint 1.00 …  signed
  finalized reserves 104000000 (latest 104000000) >= S1 53000000 + S2 50000000 + amount 1000000 =
    104000000
$ ballast bot cosign-mint 0.01 …  refused
  over-mint: finalized reserves 104000000 < S1 54000000 + S2 50000000 + amount 10000 = 104010000
$ ballast bot cosign-mint 0.01 …  refused   (asset frozen)
  I7: the breaker asset is frozen — no mint over a tripped breaker
$ ballast bot cosign-thaw  refused
  finalized block #1559 is older than the deficit block #1622: finality has not reached the drain yet
$ ballast bot cosign-thaw  signed
  finalized reserves 104000000 … >= S1 54000000 + S2 50000000 = 104000000, finalized block #1624 past
    the freeze
$ ballast bot cosign-team …  refused   the freezer would change …
$ ballast bot cosign-team …  refused   admin must stay M_admin … another admin could thaw without the
  bot …
```

ON CHAIN, V3 TESTNET

170098-1	mint 1.00 executes
170112-1	freeze, asset Frozen
170114-1	transfer refused: `AssetNotLive`
170115-1	watcher thaw refused: `NoPermission`
170133-1	gated thaw, asset Live
170134-1	transfer 1.00 Ok
170156-1	safe `set_team`, co-signed

261/261
V3 extrinsics in 23 runs (failed and errored runs kept as history), all final ≤ #261558 on 2026-10-31

22/22
role drill steps, 20/20 state checks

15/15
attestation drill steps, 6/6 checks

16/16
role setup steps, 6/6 checks

ARCHITECTURE

THE RUNTIME ENFORCES THE ROLES

10 09 08 07 06 05 04 03 02 01

PORTALDOT V3 RUNTIME · ENFORCEMENT

solid: Portaldot V3 runtime · dashed: off-chain processes · bottom: the reserve chain

Assets · asset 4401 pUSDC-test (+ control asset 4402, keyless roles)
owner = issuer = `M_mint` · admin = `M_admin` · freezer = `M_watch`

System remarks
`BALLAST/1` attest · heartbeat · freeze

Revive · ReserveOracle
`post()` attester only · `status()` · `isBacked(maxAge)` for EVM dApps
`0x4832…155e`

`freeze_asset` | `mint · set_team` | `thaw_asset`

M_watch · 1-of-2
`as_multi_threshold_1`

M_mint · 2-of-2
A, B · `as_multi`

M_admin · 2-of-3
A, A2, B · `as_multi`

`batch_all[remark, freeze]` | `A: first approval` | `B: final, if the gate passes` | `attest · heartbeat` | `reads`

watchdog
keys W1, W2 · freeze only · reads latest on each provider

operator
keys A, A2 · proposes mint, thaw, team

reserve bot
key B · co-signs mint, thaw, set_team · SQLite decision log

attester
key B · remark, then `post()` to the oracle

ballast verify
re-reads both chains; rejects wrong signers

`balanceOf @ latest, each provider` | `@ finalized and latest` | `@ finalized` | `@ block hash`

Reserve chain · ReserveVault.sol + ERC-20 reserve token
`reserves()` · `release()` owner-only · in these drills: a local anvil chain, NOT Sepolia · target: Sepolia with Circle test USDC

10 09 08 07 06 05 04 03 02 01

THE STOP LIVES IN THE ASSET

QUESTION	A RESERVE NUMBER ALONE (FRAMING, NOT MEASURED)	BALLAST, MEASURED ON V3
WHO STOPS TRANSFERS?	each dApp that chose to read it	the runtime refuses `Assets.transfer` → `AssetNotLive` (`170114-1`; other transfer paths not drilled)
CAN THE STOPPER RESTART IT?	the number carries no role on the asset	no: a watcher's thaw → `NoPermission` (`170115-1`)
WHEN CAN IT RESTART?	whenever a consumer decides	the bot co-signs a thaw only once finalized reserves pass the drain: refused at anvil #1559 < #1622, co-signed in `170133-1`. A and A2 together could still thaw without it.
CAN SOMEONE FAKE THE RECORD?	depends on who signs the feed	a forged remark from another key (`170253-1`) → `ballast verify` REJECTED; 4/4 bot attestations PASS

Built from pallet-native modules (Assets roles, Multisig, System remarks) with no contract code in the enforcement path. The one contract on Portaldot, `ReserveOracle`, is a read-only mirror: it cannot freeze or thaw anything. The reserve vault lives on the reserve chain.

PORTALDOT'S OWN MODULES

ASSETS	`create` · `set_metadata` · `set_team` · `mint` · `freeze_asset` · `thaw_asset` · `transfer` breaker asset 4401 and control asset 4402 on V3
MULTISIG	`as_multi` · `as_multi_threshold_1` · `cancel_as_multi` three role multisigs: 2-of-2, 2-of-3, 1-of-2
UTILITY	`batch_all` the freeze and its reason remark land as one extrinsic
SYSTEM	`remark_with_event` `BALLAST/1` attestations, heartbeats, freeze reasons
REVIVE	`instantiate_with_code` · `call` `ReserveOracle` on V3 (deployed in 169960-1) exposes `isBacked(maxAge)` to EVM dApps; `post()` mirrors ran on V3, the `isBacked` read only on the stand-in

PINNED TO THE LIVE CHAIN

GENESIS
`0xd4f9acbc…5283bd`

RUNTIME
`portaldot-v3 / 3002`

BLOCKS
`6000 ms`

OPEN SOURCE
MIT · one CLI, `ballast` · 342 tests (301 pytest, 20 Foundry, 21 Hardhat)

10
09
08
07
06
05
04
03
02
01

WHO NEEDS A LOAD LINE

The first issuer of a dollar on Portaldot, who needs a safety layer it can show, and that the ecosystem can require of any issuer.

The dApp builder deciding whether checkout may accept that dollar, who must tell users why it is safe.

PATH · STEPS 2 AND 3 ARE PLANS

1

TESTNET, TODAY

V3 drills pass; the Sepolia reserve is drilled with real finality; one person holds every key.

2

OUTSIDE WATCHERS

a watcher run by someone other than us; W2 on GitHub Actions is still ours.

3

A REAL ISSUER

watcher and attester keys held by parties other than the issuer; community-watcher mode.

Funding (plan): an ecosystem grant to reach a deployment with outside watchers; after that, the dollar's issuer runs Ballast as the safety layer the ecosystem could ask of it. **Today:** no external users and no revenue.

DONE ON V3. NEXT: THE FINAL

DONE

Role drill on V3: PASS `22/22 + 20/20` · attestation drill PASS `15/15 + 6/6`

Sepolia vault (100.00 test USDC) drilled with the real finality wait: PASS `8/8 + 18/18`

Outage, heartbeat and governance drills on V3 (scratch assets); W2 froze via GitHub Actions with W1 off

`261/261` V3 extrinsics (23 runs) final · `35` freeze drills benchmarked · reserves page live

`342` tests · `232` commits from 2026-10-14 to build `03458f0`

NEXT

A watcher run by someone other than us (the Actions cron fired once in ~5 h)

`ReserveOracle` v2 with `trip()` on V3: its deposit exceeds our keys (stand-in only)

A community watcher with an outside operator (drilled on our own anvil)

Keys held by parties other than the issuer

A fresh oracle mirror against the Sepolia reserve (the last post is 2026-10-24, local anvil)

Scan for the repository.

REPOSITORY
`github.com/edycutjong/portaldot-hackathon-2026-ballast-edycu`

V3 PROOF TABLES
`docs/PROOF.md`

THE FREEZE, IN THE NODE CONSOLE
`block #170,112`

AI assistance: built with Claude Code (Anthropic). Most of the code, the tests and the prose were drafted with AI assistance and reviewed by the author. The architecture, the role map and the on-chain design decisions are the author's.